Pop-up Questions Context-Free Parsing (E. Ch. 10)

James Pustejovsky – CS115B

Quick-Fire Questions

Designed for random cold-calling based on the weekly reading.

Q1. What is the standard output of a parser?
Answer: A tree (specifically, a parse tree). Such trees answer basic questions of "who-did-what-to-whom" and are used in downstream tasks like semantic analysis.

Q2. How does the number of possible parse trees grow in relation to the length of the sentence?
]**Answer:** It grows super-exponentially.

Q3. What is the Catalan number (C_n)?
Answer: It is the number of possible bracketings (or derivations) for a string in a minimal context-free grammar. It is defined by the formula:

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

Q4. The standard CKY algorithm requires the grammar to be in what specific form?
Answer: Chomsky Normal Form (CNF). This means productions must be binary (two non-terminals) or unary (one terminal).

Q5. In terms of the sentence length M and number of productions R, what is the time complexity of the CKY algorithm?
Answer: $\mathcal{O}(M^3R)$ (Cubic in sentence length).

Q6. "I shot an elephant in my pajamas" is a classic example of what specific type of ambiguity?
Answer: Attachment ambiguity. The prepositional phrase "in my pajamas" can attach to the verb "shot" or the direct object "elephant".

Q7. When evaluating a parser using PARSEval, what is the difference between "labeled" and "unlabeled" precision?
Answer: Labeled precision requires matching the constituent's **phrase type** (e.g., NP, VP), while unlabeled precision only requires matching the **constituent structure** (the span of words).

Q8. In the CKY algorithm, if we are filling the table for a sentence of length M, what does the cell $t[0, M]$ represent?
Answer: The set of non-terminals that can derive the **entire input string**. If the start symbol S is in this cell, the string is in the language.

What is Super-Exponential Growth?

Super-exponential growth means a function grows faster than any exponential function. To put this in perspective, consider three growth rates:

Polynomial growth (e.g., n^2, n^3) — manageable, the basis of "efficient" algorithms

Exponential growth (e.g., 2^n) — already very fast; doubling the input squares the output

Super-exponential growth (e.g., $n!$ or the Catalan numbers) — faster still; these functions eventually outpace *any* exponential

Catalan Numbers

The n-th Catalan number is given by:

$$C_n = \frac{(2n)!}{(n+1)!\,n!}$$

The factorial in the numerator is what drives the super-exponential behavior. In the context of parsing, C_n counts the number of distinct binary bracketings of a sequence of $n+1$ elements—equivalently, the number of full binary trees with $n+1$ leaves. Thus, a sentence of m words has C_{m-1} possible bracketings.

n	C_n	Bracketings for $n+1$ words
1	1	2 words
2	2	3 words
3	5	4 words
4	14	5 words
9	4,862	10 words
19	$\approx 1.7 \times 10^9$	20 words

Table 1: Catalan numbers and their relationship to sentence length.

Implications for Parsing

The practical implication is that one simply cannot enumerate all possible parse trees for even moderately long sentences—the search space explodes too quickly. This is precisely why dynamic programming algorithms like CKY are essential: they avoid redundant computation by reusing shared substructures, bringing the complexity down to $O(n^3)$, which is polynomial.

The CKY Algorithm

The core of the CKY algorithm consists of three nested loops:

```
1: for ℓ ∈ {2, 3, ..., M} do                          ▷ Iterate over constituent lengths
2:     for m ∈ {0, 1, ..., M − ℓ} do                   ▷ Iterate over left endpoints
3:         for k ∈ {m + 1, m + 2, ..., m + ℓ − 1} do    ▷ Iterate over split points
4:             for (X → Y Z) ∈ R do                    ▷ Iterate over rules
5:                 Check if Y ∈ t[m, k] and Z ∈ t[k, m + ℓ]
6:             end for
7:         end for
8:     end for
9: end for
```

Counting Iterations

Let us count the number of times the innermost check is executed.

Loop 1: Span length ℓ. This runs from 2 to M, so $M - 1 = O(M)$ iterations.

Loop 2: Left endpoint m. For a given ℓ, this runs from 0 to $M - \ell$, giving $M - \ell + 1$ iterations.

Loop 3: Split point k. For a given span $[m, m+\ell]$, the split point k ranges over $\{m+1, \ldots, m+\ell-1\}$, giving $\ell - 1$ iterations.

Loop 4: Grammar rules. This iterates over all $|R|$ rules.

Total Count

The total number of operations is:

$$|R| \cdot \sum_{\ell=2}^{M} \sum_{m=0}^{M-\ell} (\ell - 1)$$

The inner sum (over m) has $M - \ell + 1$ terms, each equal to $\ell - 1$:

$$\sum_{m=0}^{M-\ell} (\ell - 1) = (M - \ell + 1)(\ell - 1)$$

So the total is:

$$|R| \cdot \sum_{\ell=2}^{M} (M - \ell + 1)(\ell - 1)$$

Substituting $j = \ell - 1$ (so j runs from 1 to $M - 1$):

$$|R| \cdot \sum_{j=1}^{M-1} (M - j) \cdot j = |R| \cdot \sum_{j=1}^{M-1} (Mj - j^2)$$

Using standard summation formulas:

$$\sum_{j=1}^{M-1} j = \frac{(M-1)M}{2}$$

$$\sum_{j=1}^{M-1} j^2 = \frac{(M-1)M(2M-1)}{6}$$

Therefore:

$$|R| \cdot \left[M \cdot \frac{(M-1)M}{2} - \frac{(M-1)M(2M-1)}{6} \right]$$

$$= |R| \cdot \frac{(M-1)M}{6} \left[3M - (2M-1) \right]$$

$$= |R| \cdot \frac{(M-1)M(M+1)}{6}$$

Conclusion

The total number of operations is:

$$\frac{|R| \cdot (M-1) \cdot M \cdot (M+1)}{6} = O(M^3 \cdot |R|)$$

Since $|R|$ (the number of grammar rules) is typically treated as a constant independent of sentence length, the time complexity of CKY is $O(M^3)$—**polynomial**, in contrast to the super-exponential C_{M-1} number of possible parses.